

Mastering Azure AKS - Hands-On

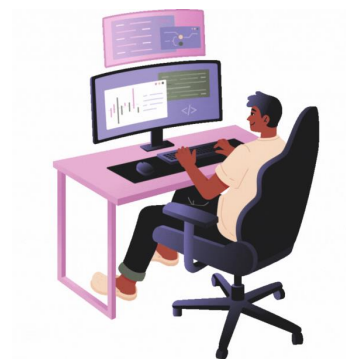




Mastering Azure AKS - Hands-On



AKS is a fully managed Kubernetes offering from Azure that simplifies container orchestration at scale



Cluster Creation

Node Groups

Load Balancing

Storage

Scaling



**The Ultimate Linux Bootcamp for
DevOps SRE & Cloud Engineers**



**Practical Kubernetes –
Beyond CKA and CKAD**



**Automation with Ansible -
Hands-on DevOps**



**Kubernetes and
Cloud Native Associate**



**Argo CD for the Absolute
Beginners - Hands-On**



**Mastering Docker Essentials
- Hands-on**



**Kulbhushan
Mayer**



Yogesh Raheja

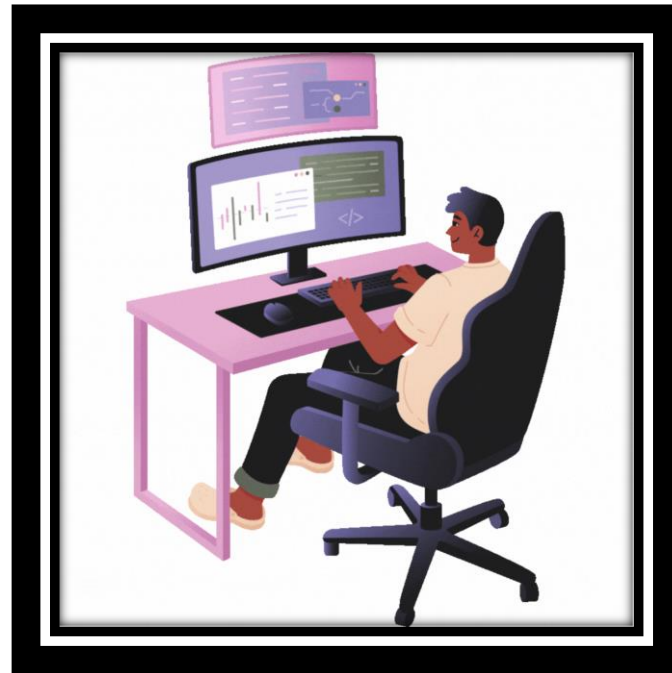
How does this course work?



How does this course work?



Core Concepts



Hands-On Demonstrations



Assessments



**Container
Orchestration**



**How Azure AKS
solves those
challenges**



**Installing CLI tools
Launching first AKS
cluster**



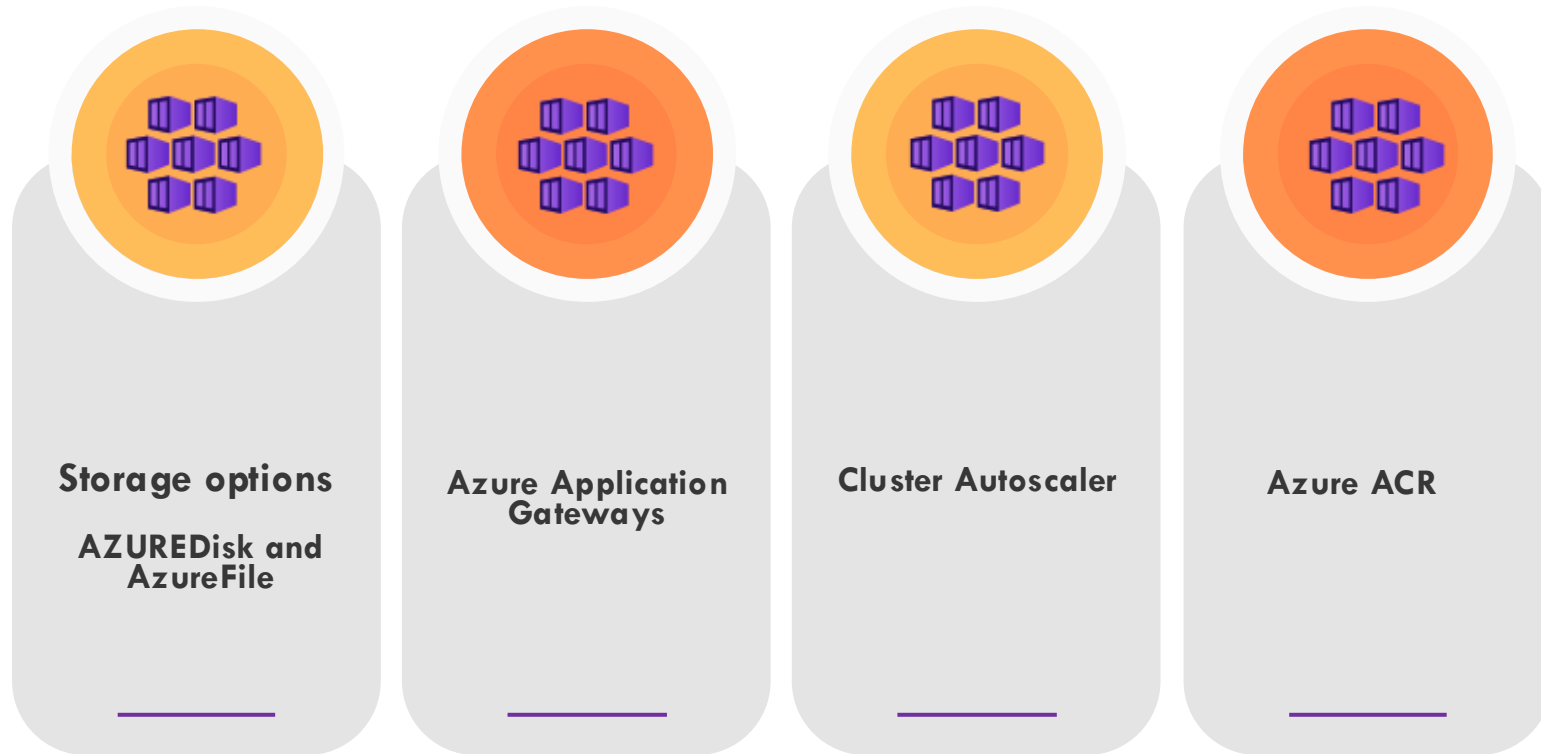
Upgrade AKS cluster



**Explore worker
node options
Managed Node Pools**



**Load balancing
mechanisms**



Real-World Demonstrations

Architectural Insights

Practical Use Cases

Explore Our In-depth Courses



Build and Scale with AWS Cloud - A
Hands-On Beginners Guide



Practical Kubernetes – Beyond CKA
and CKAD | Hands-On

Introduction to AZURE AKS





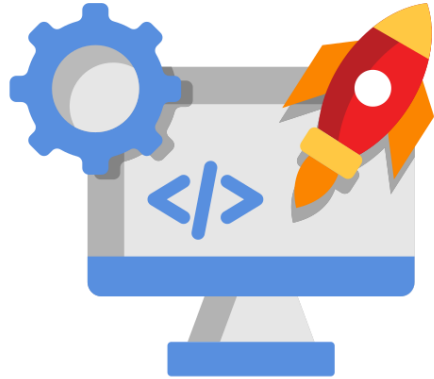
Introduction to AZURE AKS

Section Overview

- *Fundamentals of Container Orchestration*
- *Challenges of Managing Kubernetes at Scale*
- *Introduction to Azure AKS (Azure Kubernetes Service)*

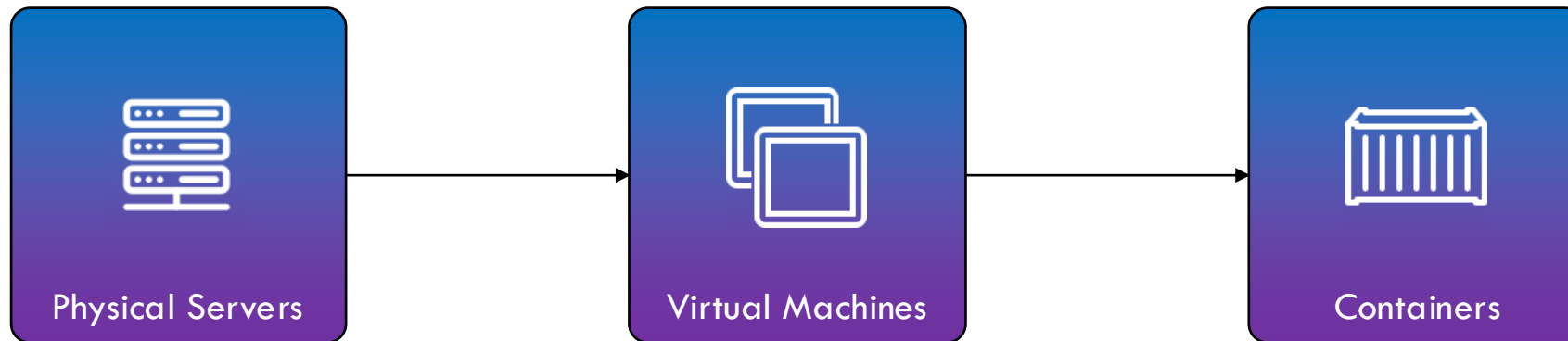
Demystifying Container Orchestration

Demystifying Container Orchestration



Why container orchestration is essential ?

Demystifying Container Orchestration



Containers



- Faster development and deployment
- Consistent application behavior across environments
- Portability: build anywhere, run anywhere (laptop, VM, cloud, bare metal)
- Smooth transitions between testing, staging, and production

Advantages of Containers



- Faster startup times, often in just a few seconds
- Lower resource usage, since containers don't need a full guest operating system
- Greater portability, making them easy to move across platforms without modification

Containers are a great way to package and run applications

The Production-Scale Problem

- Real-world deployments just run one or two containers
- Organizations often run hundreds to thousands of containers
- Containers are distributed across multiple servers and regions
- Microservices architecture adds further complexity
- Applications are broken into small, independent services
- Services communicate with each other across the network
- Manual container management becomes impractical
- High risk of human error and operational inefficiency



Key Challenges in Managing Containers at Scale

Automatic scaling

Self-healing

Load balancing

Rolling updates and rollbacks

High availability

Data integrity

Service discovery

Persistent storage

Security

Container Orchestration



- Solved the packaging and portability problems
- Didn't solve the coordination problem

Container Orchestration

Container Orchestration

Container Orchestration

- Deploying containers across multiple nodes
- Automatically scaling workloads based on traffic
- Replacing failed containers to ensure high availability
- Distributing traffic efficiently across services
- Managing networking, updates, and rollbacks - often with zero downtime



Why Kubernetes?



- A powerful container orchestration platform
- Used to deploy, scale, and manage containerized applications
- Creates self-contained environments for applications
- Integrates compute, networking, storage, and configuration

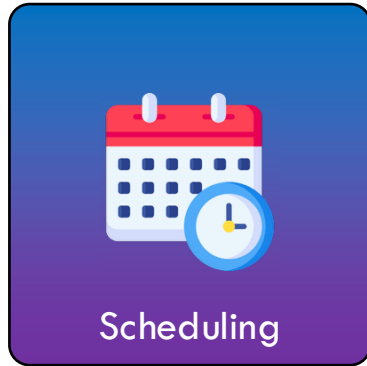


Why Kubernetes?

- Open source and cloud-agnostic
- Capable of running in any environment - on-premises, cloud, or hybrid
- Based on declarative configuration, making it easy to define your app's desired state
- Integrated with modern DevOps pipelines and CI/CD workflows
- Supported by a large and active open-source community



Kubernetes Features at a Glance



- Core Function of Kubernetes
- Resource-Based Decisions
- Policy & Constraint Driven
- Efficient Cluster Utilization
- Workload Balancing

Kubernetes Features at a Glance



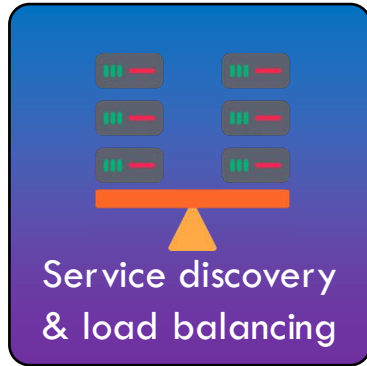
- Automatic Restart
- Pod Rescheduling
- Minimized Downtime
- Built-in Resilience

Kubernetes Features at a Glance



- Auto-scaling Triggers
- Efficient Resource Utilization

Kubernetes Features at a Glance



- Built-in DNS service for internal communication
- Containers communicate using simple service names
- Automatic traffic distribution across healthy pods
- Eliminates need for manual IP address management

Kubernetes Features at a Glance



- Gradual deployment of new application versions
- Minimizes service disruption during updates
- No Downtime Deployment
- Automatic Rollbacks

Kubernetes Features at a Glance



- Inject environment-specific data at runtime
- Manage sensitive data like passwords and API keys securely

Kubernetes Features at a Glance



- Automatically Mount Storage Volumes
- Supports Local Storage, Public Cloud Providers, and NFS
- Simplifies Persistent Data Management

Kubernetes Features at a Glance



- Custom controllers in Kubernetes
- Extend Kubernetes functionality
- Ideal for complex, stateful applications (e.g., databases)
- Automate routine operational tasks
 - Provisioning
 - Scaling
 - Replication
 - Failover
- Work based on declared user intent



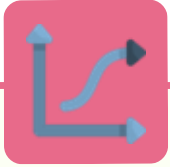
Move faster, scale confidently, and run applications anywhere

Limitations of Native Kubernetes

Limitations of Native Kubernetes



Key Limitations



Steep Learning Curve

- Kubernetes setup and management require advanced skills in networking, RBAC, storage, and security - challenging for beginners



Manual Cluster Management

- Tasks like installation, scaling, and patching are often manual, leading to errors and downtime risks



Security Requires Extra Effort

- Enterprise-grade security needs custom setups and external tools beyond Kubernetes' built-in capabilities



Fragmented Tooling

- No built-in solutions for monitoring, logging, or CI/CD - teams must integrate and maintain separate tools

Enterprise Kubernetes Platforms



Built on top of native Kubernetes but include enhanced tooling, security features, automation, and vendor support, making it easier and safer to use in large companies



Red Hat
OpenShift



vmware® Tanzu



RANCHER
BY SUSE

Managed Kubernetes Service



Managed Kubernetes services reduce the complexity of deploying, managing, and scaling Kubernetes at production scale



*Amazon Elastic
Kubernetes Service*



*Google Kubernetes
Engine*



*Azure Kubernetes
Service*

Handles infrastructure and Kubernetes management, so you can focus on your applications

Some Leading Managed Kubernetes Services



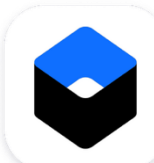
Amazon Elastic Kubernetes Service



Azure Kubernetes Service



Google Kubernetes Engine



IBM Cloud Kubernetes Service



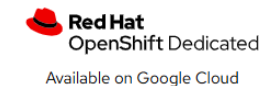
Oracle Container Engine for Kubernetes (OKE)



Alibaba Cloud ACK (Container Service for Kubernetes)



Platform9 Managed Kubernetes



What is Azure AKS?

- Limitations of Native Kubernetes
- Managed Kubernetes services



What is AZURE AKS?

- Fully managed, certified Kubernetes-conformant service
- Compatible with existing Kubernetes tools and manifests
- Enables easy workload migration across environments
- Eliminates the need to re-architect applications
- Simplifies Kubernetes setup and operations at scale
- Provides production-grade clusters with minimal effort
- Azure manages the control plane, availability, and security



Users can focus on deploying and running containerized apps

Why Choose AKS?



- ❌ Install
- ❌ Operate
- ❌ Upgrade



Kubernetes control plane



Multiple Availability Zones for high availability and resilience

- For Compute layer (Data plane)
 - Use self-managed virtual machine nodes for total control
 - Choose AKS Auto Mode managed nodes for simplified operations
 - Go serverless with Azure virtual nodes, without managing any infrastructure

Features of AKS



Built-in High Availability

Multiple AZs, automatically scaling and replacing unhealthy nodes, Stay highly available and resilient - even during failures



Integrated Security and Access Control

Integrates with Azure IAM and Kubernetes RBAC, Works with Azure Key Vault



Autoscaling and Flexibility

Kubernetes-native tools like Cluster Autoscaler, or modern solutions like Karpenter, AKS automatically provisions and scales infrastructure



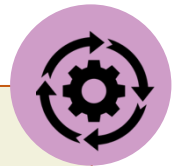
Persistent Storage Made Easy

AKS supports native Persistent Volumes (PVs and PVCs) using Azure Disk, Azure files via CSI drivers



Observability and Monitoring

Azure Monitor Container Insights, and native Prometheus, Azure Distro for OpenTelemetry (ADOT)



Easy Management Interfaces

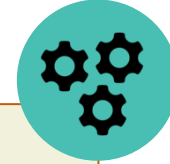
Azure Console, Azure CLI, ARM, Terraform

Features of AKS



Hybrid and Edge Support

Run AKS clusters in your on-prem data center using the same components as Azure-hosted clusters. And with AKS on Outposts, you can extend Kubernetes to edge locations



Seamless Integration with Amazon ECR

Integrates directly with Azure ACR, a fully managed container registry that provides secure, scalable, and high-performance image storage



Introduction to Azure AKS

Section Summary

- *Importance of container orchestration*
- *Limitations of native Kubernetes*
- *Azure AKS (Azure Kubernetes Service)*
- *Key features of Azure AKS:*
 - *High availability*
 - *Autoscaling*
 - *Enhanced security*
 - *Observability*
 - *Integration with Azure tools (e.g., ACR)*

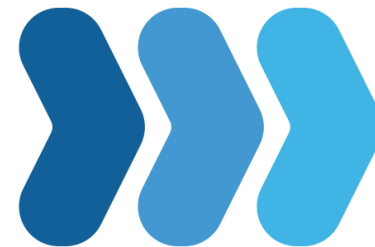
Getting Started with Azure AKS





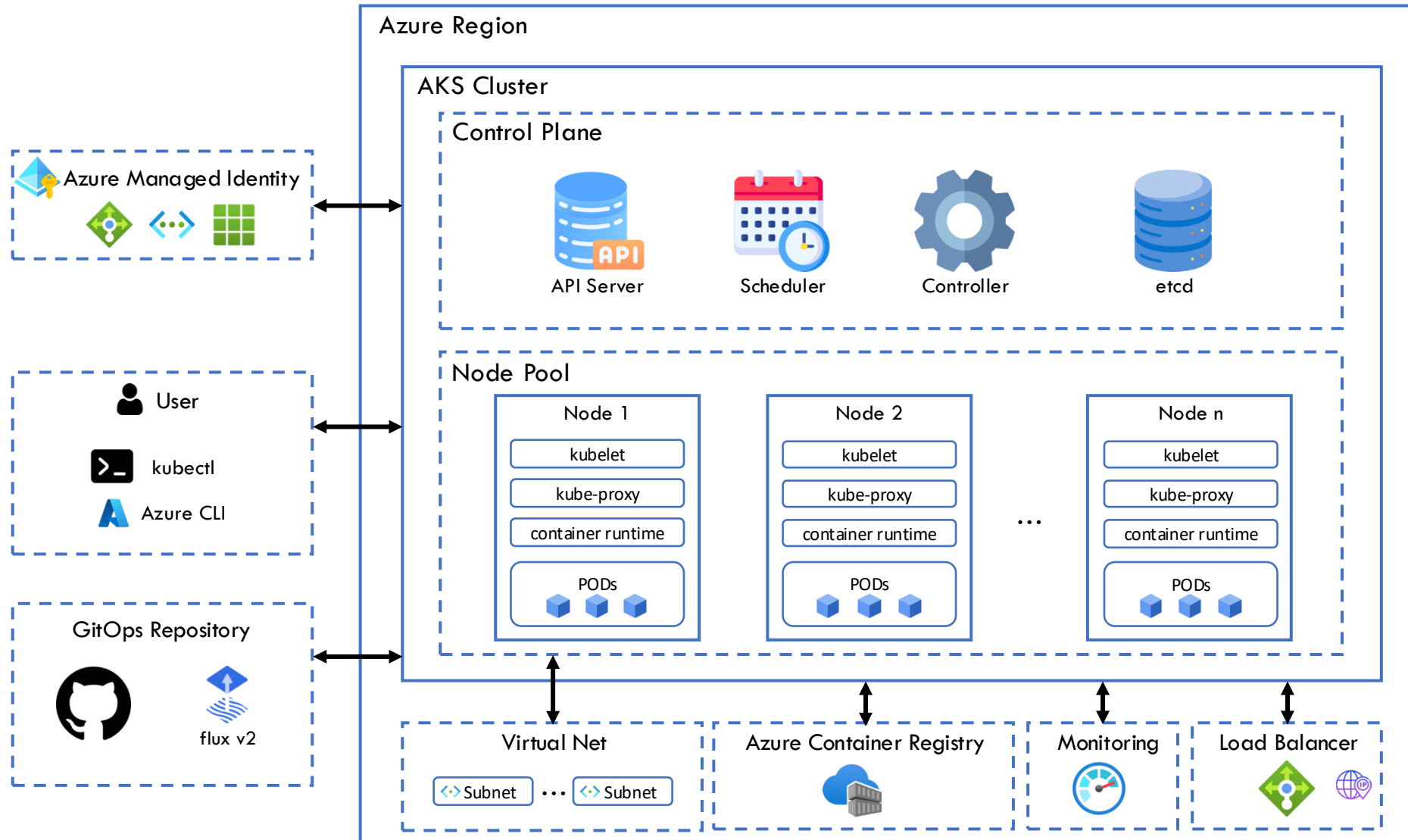
Creating an AKS Cluster

Section Overview

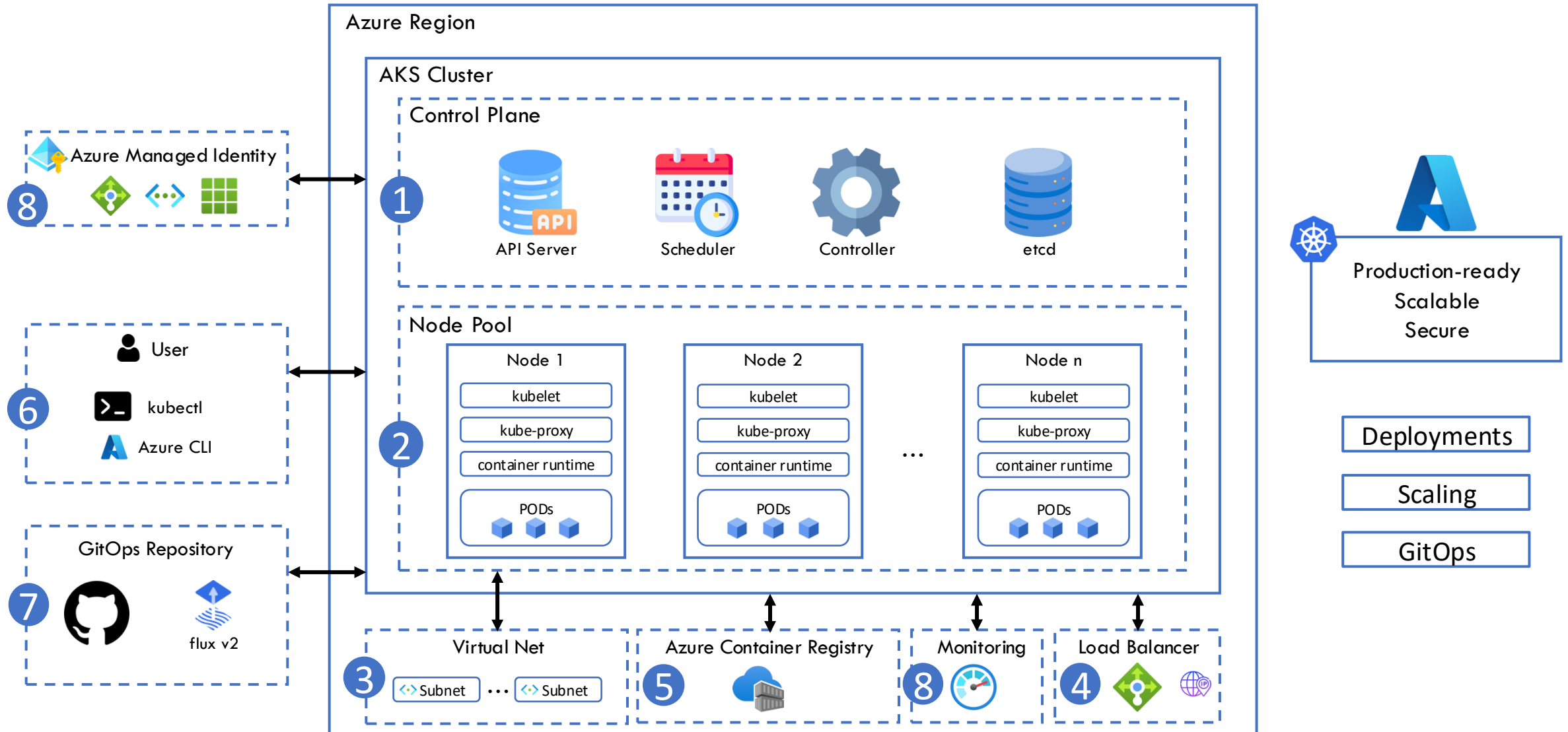


Understanding AKS Architecture *(Control Plane, Node Pools, Azure Integration)*

Understanding AKS Architecture



Understanding AKS Architecture



Creating an AKS Cluster





Creating an AKS Cluster

Section Overview

- Explore how Kubernetes scales worker nodes automatically using the **Cluster Autoscaler** in Amazon EKS
- Learn:
 - ☐ Working
 - ☐ IAM role set up
 - ☐ Deployment using auto-discovery
- Watch nodes scale based on demand

Understanding Cluster Autoscaler for Node Groups

Understanding Cluster Autoscaler for Node Groups

Understanding Cluster Autoscaler for Node Groups

- Just setting ASG min, max, and desired values doesn't enable autoscaling based on pod demand
- By default, ASG maintains the desired EC2 count
- It replaces unhealthy instances but doesn't scale for pod demand
- Scaling up/down needs the **Cluster Autoscaler**

Demo

Creating Azure Resource Group



Demo

Creating Virtual Network



Demo

Creating Log Analytics Workspace



Demo

Creating Azure Container Registry



Demo

Create AKS Cluster



Demo

AKS Cluster Walkthrough in Azure Portal



Demo

Installing Azure CLI, kubectl



Demo

Configuring AKS Access



Demo

Upgrading the AKS Cluster Kubernetes Version



Demo

Creating and Managing AKS User Node Pools



Demo

*Deploying a Pod to Newly
added NodePool using
taints & NodeSelectors*



Demo

Deleting NodePool



Creating an AKS Cluster



Azure

Demo

Create Namespace for Demo Application



Demo

Creating Resources for Redis for Demo Application



Demo

Creating Deployment for Participate Application



Demo

*Expose Participate
using ServiceType
LoadBalancer*



Demo

Creating Resources for Postgres for Demo Application



Demo

Creating Deployment to Sync Redis entries to Postgres



Demo

Creating Resources for Submissions Application



Demo

Enabling Azure Application Gateway as Ingress Controller



Demo

Creating Ingress for Participate and Submissions Applications



Demo

Creating Resources for Pick Winner Application



Demo

Cleaning up Application Gateway Resources



Storage Options



Azure

Kubernetes Storage: StorageClass, CSI Driver, PV, PVC, and Pod Integration

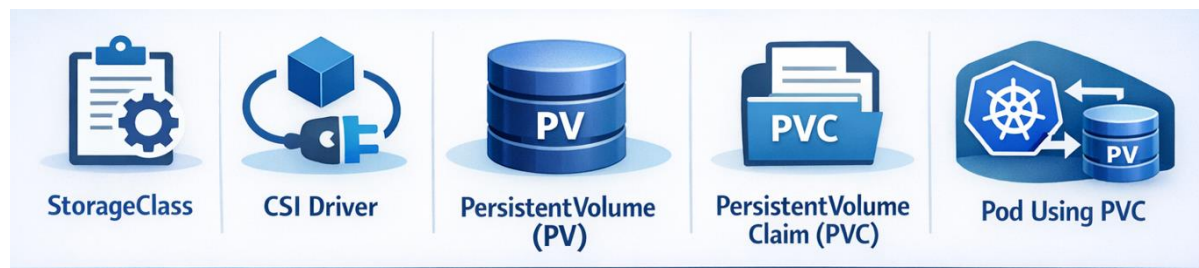


Introduction

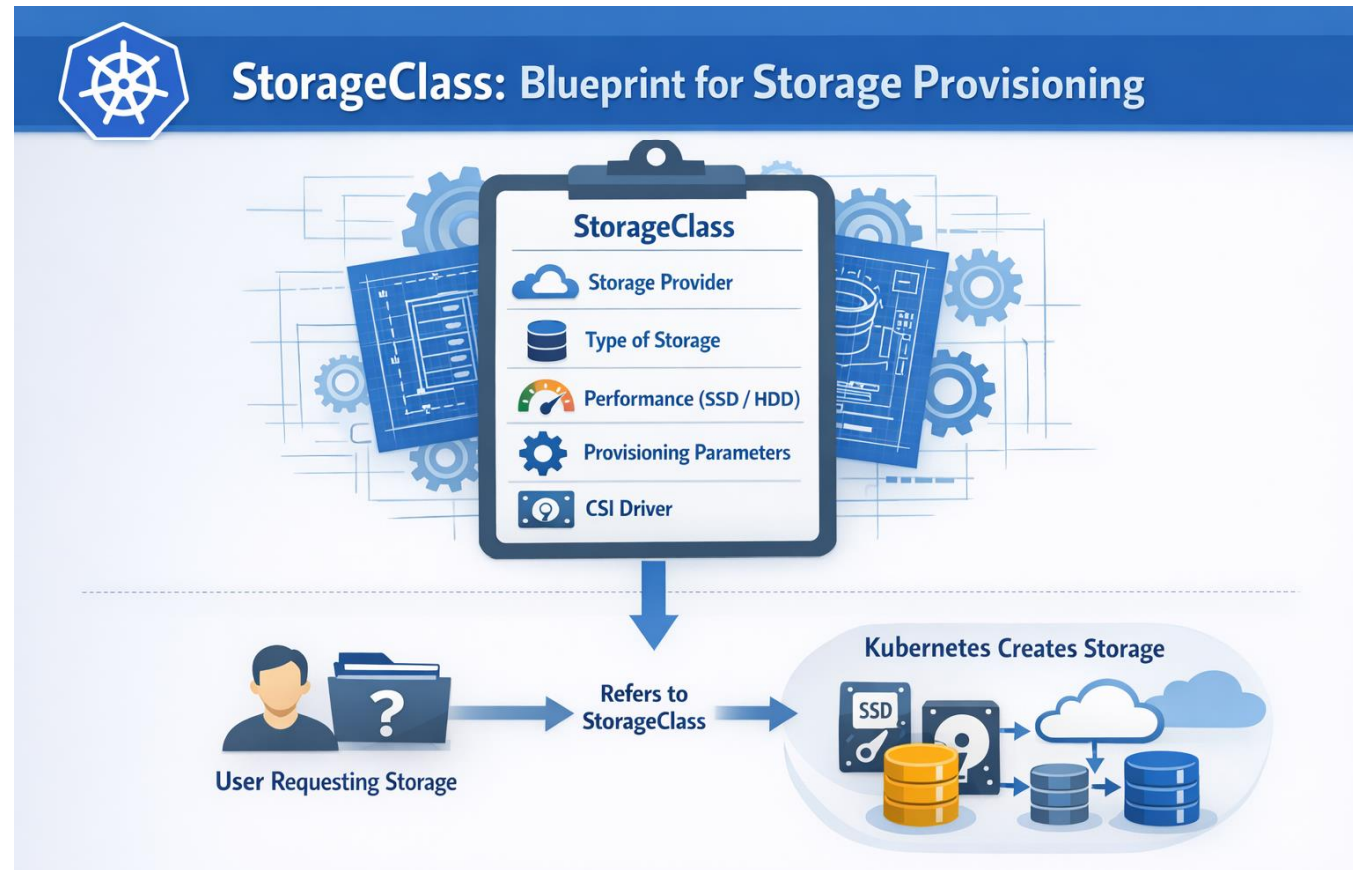
Section Overview

- *How storage works in Kubernetes*
- *Persistent storage options in AKS*

Key Components:



StorageClass – Defines the Storage Blueprint



StorageClass Example in AKS

```
kubectl get storageclass
```

Output:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
azurefile	file.csi.azure.com	Delete	Immediate	true	109m
azurefile-csi	file.csi.azure.com	Delete	Immediate	true	109m
azurefile-csi-premium	file.csi.azure.com	Delete	Immediate	true	109m
azurefile-premium	file.csi.azure.com	Delete	Immediate	true	109m
default (default)	disk.csi.azure.com	Delete	WaitForFirstConsumer	true	109m
managed	disk.csi.azure.com	Delete	WaitForFirstConsumer	true	109m
managed-csi	disk.csi.azure.com	Delete	WaitForFirstConsumer	true	109m
managed-csi-premium	disk.csi.azure.com	Delete	WaitForFirstConsumer	true	109m
managed-premium	disk.csi.azure.com	Delete	WaitForFirstConsumer	true	109m

StorageClass Example in AKS

➤ Common AKS StorageClasses

- managed or managed-csi, which is also configured as default storageclass for AKS
 - Uses Azure Managed Disk
 - Standard SSD disk
 - Default StorageClass
- managed-premium
 - Uses Premium SSD disk
 - Higher performance

StorageClass Example in AKS

➤ Common AKS StorageClasses

- azurefile or azurefile-csi
 - Uses Azure File Share
 - Supports shared storage across multiple pods
- StorageClass YAML

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: managed-csi
provisioner: disk.csi.azure.com
parameters:
  skuName: StandardSSD_LRS
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

CSI Driver – The Storage Provisioning Engine

CSI stands for **Container Storage Interface**

- **CSI driver** is responsible for:
 - ✓ Creating storage in external systems
 - ✓ Attaching storage to nodes
 - ✓ Mounting storage inside pods
 - ✓ Managing lifecycle of storage
- **CSI driver** connects Kubernetes with actual storage systems like:



PersistentVolumeClaim (PVC) – Request for Storage

- **PVC** specifies:
 - Storage size
 - Access mode, (whether is ReadOnly, ReadWrite and at a time how many pods can use it)
 - StorageClass to use like default, azurefile, managed or anyother

PVC does NOT create storage directly

PersistentVolumeClaim (PVC) – Request for Storage

- PVC YAML

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: managed-csi
  resources:
    requests:
      storage: 1Gi
```

PersistentVolume (PV) – Actual Storage Resource

PersistentVolume is the actual storage created in the storage system

PV is created automatically by **CSI driver** when PVC is created

- **PV** represents:
 - Azure Managed Disk
 - Azure File Share
 - Other storage system

Discussion the flow



Pod Uses Storage via PVC

Pod does not directly use PersistentVolume

Pod uses PersistentVolumeClaim

PVC acts as bridge between Pod and storage

Pod Uses Storage via PVC

- Pod YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - mountPath: "/data"
      name: storage-volume
  volumes:
  - name: storage-volume
    persistentVolumeClaim:
      claimName: my-pvc
```

Demo

Creating PersistentVolumeClaim using azurefile



Demo

Update Deployments to use PVC



Demo

Setup to use AzureDisk as PV



Demo

Creating ConfigMap for Postgres Replication



Demo

*Update Secret with
pgpool password*



Demo

Update Postgres to headless



Demo

*Update Postgres
StatefulSet with
claimTemplates
initContainers*



Demo

*Connect with postgres
statefulset pods*



Demo

Create Deployment & Service for pgpool



Demo

*Update postgres-
connection-details with
pgpool servicename*



Scaling Node Pools



Demo

Manually Scaling NodePool



Demo

Observing NodePool Autoscaling



ACR Integration





Introduction

Section Overview

- *We explored scaling in Azure Kubernetes Service (AKS)*
- *Covered both manual scaling and autoscaling features*
- *AKS dynamically adjusts compute capacity based on workload demands*
- *Helps maintain performance while optimizing cost*
- *We will use Azure Container Registry (ACR)*
- *ACR is Microsoft Azure's managed container registry service*
- *It integrates seamlessly with AKS*

ACR Integration

Kubernetes deployments rely on container images



SECURE



SCALABLE



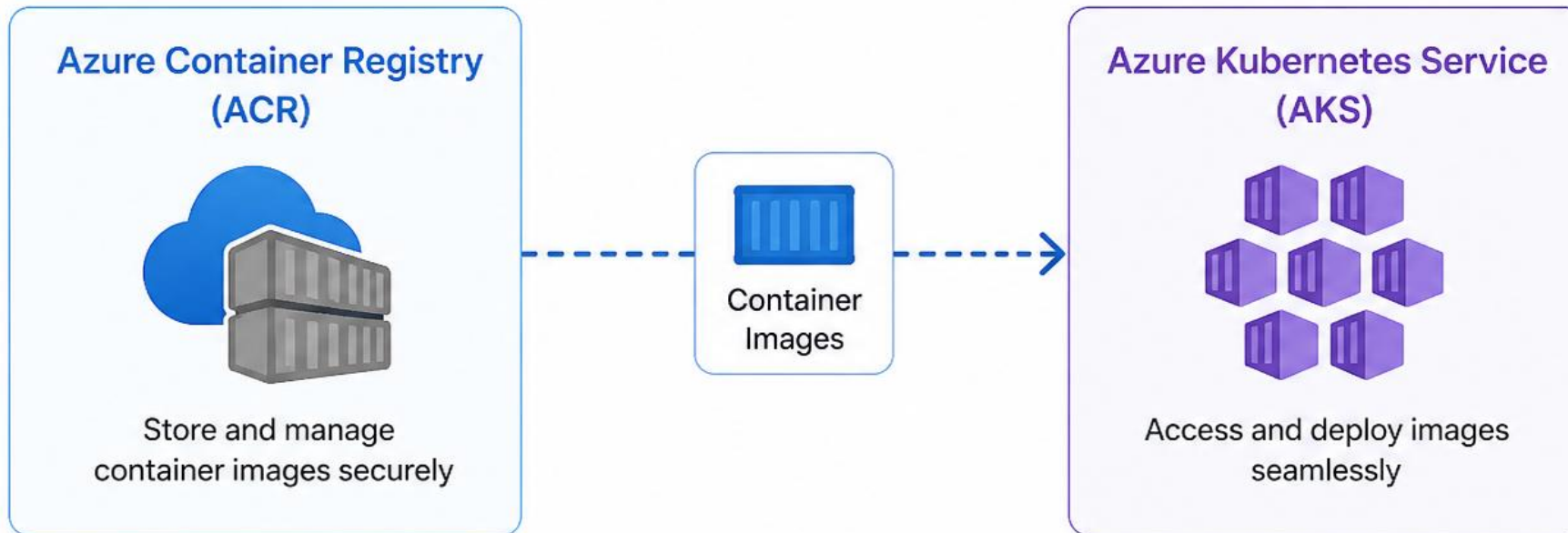
EASILY ACCESSIBLE

ACR Integration



- Creating and managing an Azure Container Registry
- Authenticate ACR from our local machine
- Pushing container images to ACR
- Deploy applications on AKS using images stored in ACR

ACR Integration



Demo

Creating ACR (Azure Container Registry)



Demo

Authenticating to ACR



Demo

Pushing Images to ACR



Demo

Attaching ACR to AKS



Demo

*Deploying in AKS using
Image from*



GitOps in AKS



Demo

Enabling GitOps



Demo

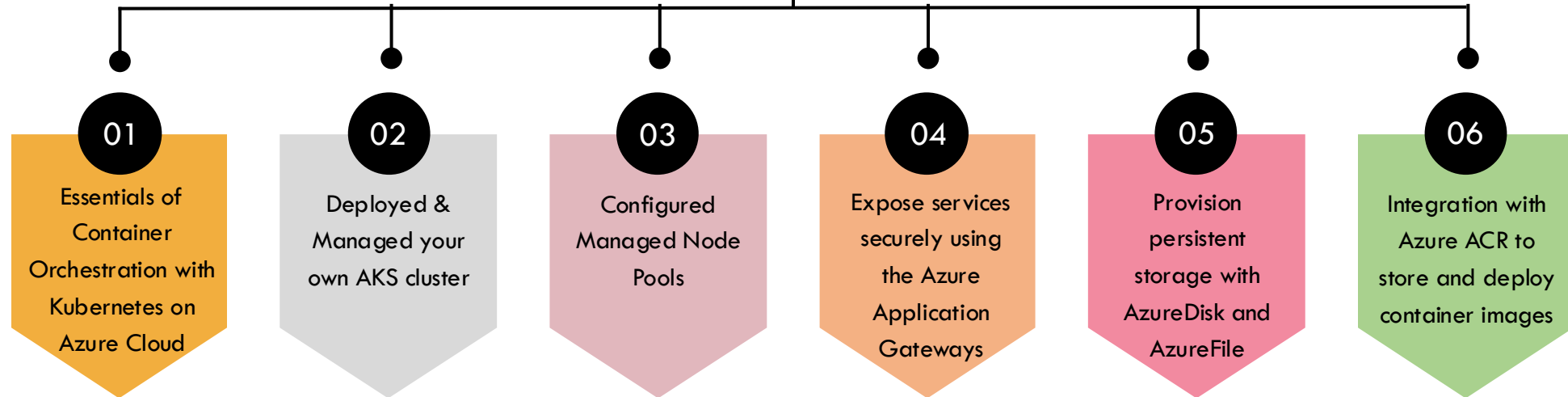
Observing end-to-end *GitOps in Action*





Mastering Azure AKS - Hands-On





Explore Our In-depth Courses



Practical Kubernetes – Beyond CKA
and CKAD | Hands-On



Mastering AWS EKS - Hands-On



Follow us on:



@thinknyx



@thinknyx



@thinknyx-technologies



@thinknyx



@thinknyx-technologies